MOUNT KENYA UNIVERSITY

DEPARTMENT OF INFORMATION TECHNOLOGY

UNIT CODE: BIT4138

UNIT NAME: ADVANCED CRYPTOGRAPHY

LOGBOOK

STUDENT NAME: Samwel Abuga

Admission Number: BSCC5120241/56213

COURSE: computer science

LECTURER: Mr. Nyoro

SEMESTRER/ACADEMIC YEAR: Sem 2 year 3,

PROJECT TITTLE: SECURE FILE ENCRYPTION SYSTEM

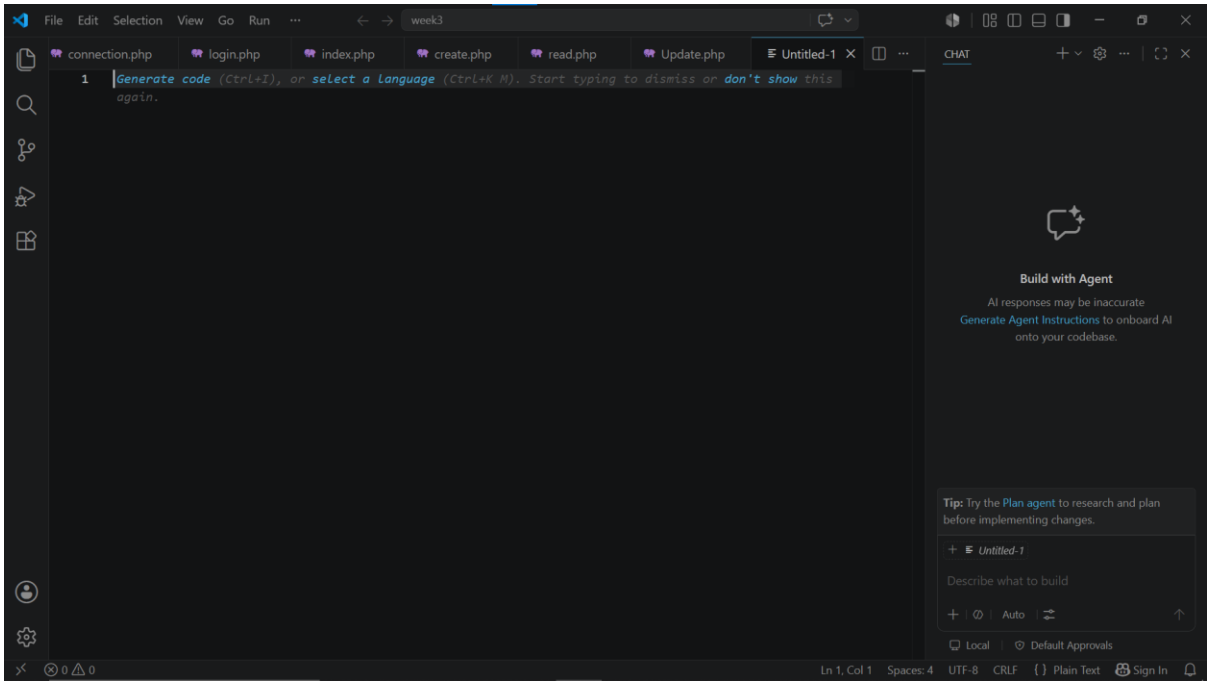
TECHNOLOGY USED: Python + Cryptography Library

GITHUB LINK:

WEEK 1:

INSTALLATION AND CONFIGURATION OF CRYPTOGRAPHY DEVELOPMENT ENVIRONMENT

Fig 1: Installation of Python and VS Code



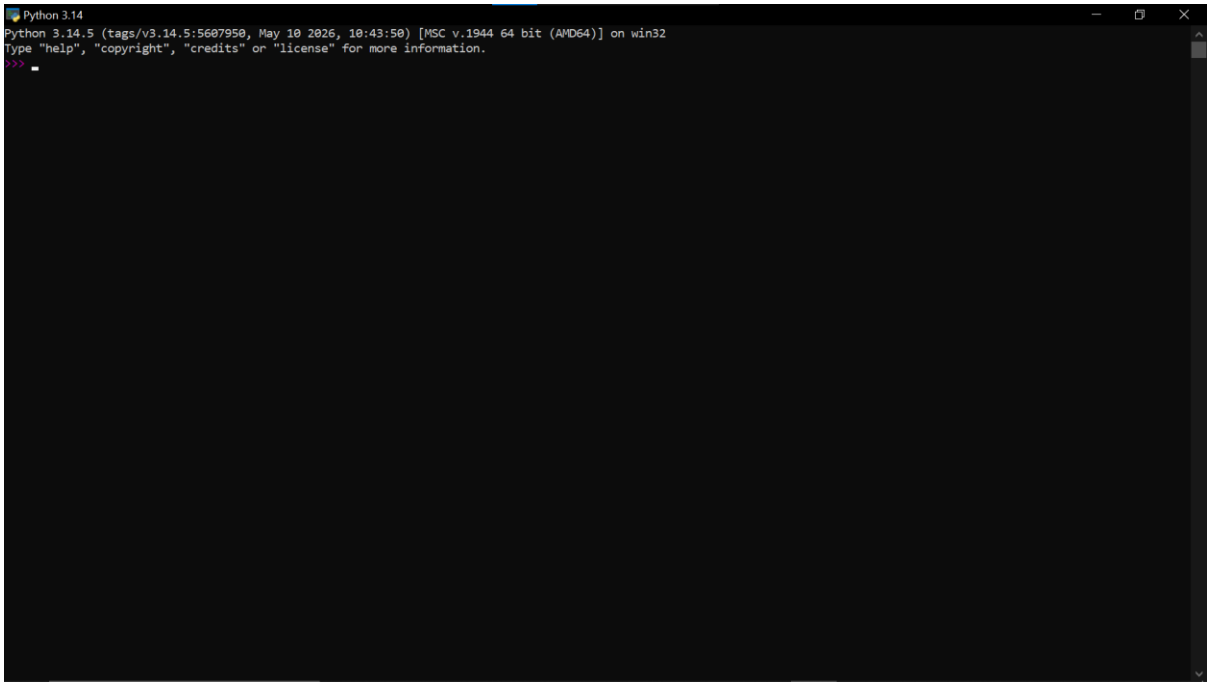


Fig 2: Installation of Cryptography Libraries

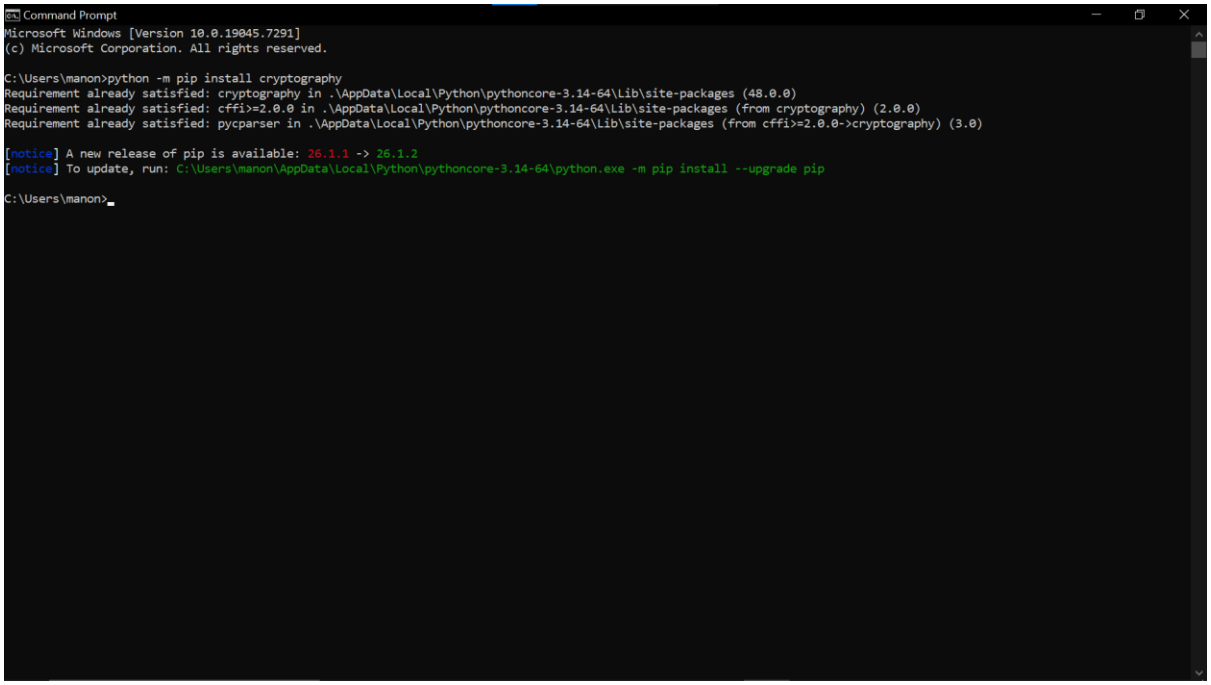


Fig 3: OpenSSL Installation Test

Fig 3: Successful verification of OpenSSL version confirming successful installation and configuration for cryptographic key generation and certificate management.

Fig 4: Local Encryption Script Execution

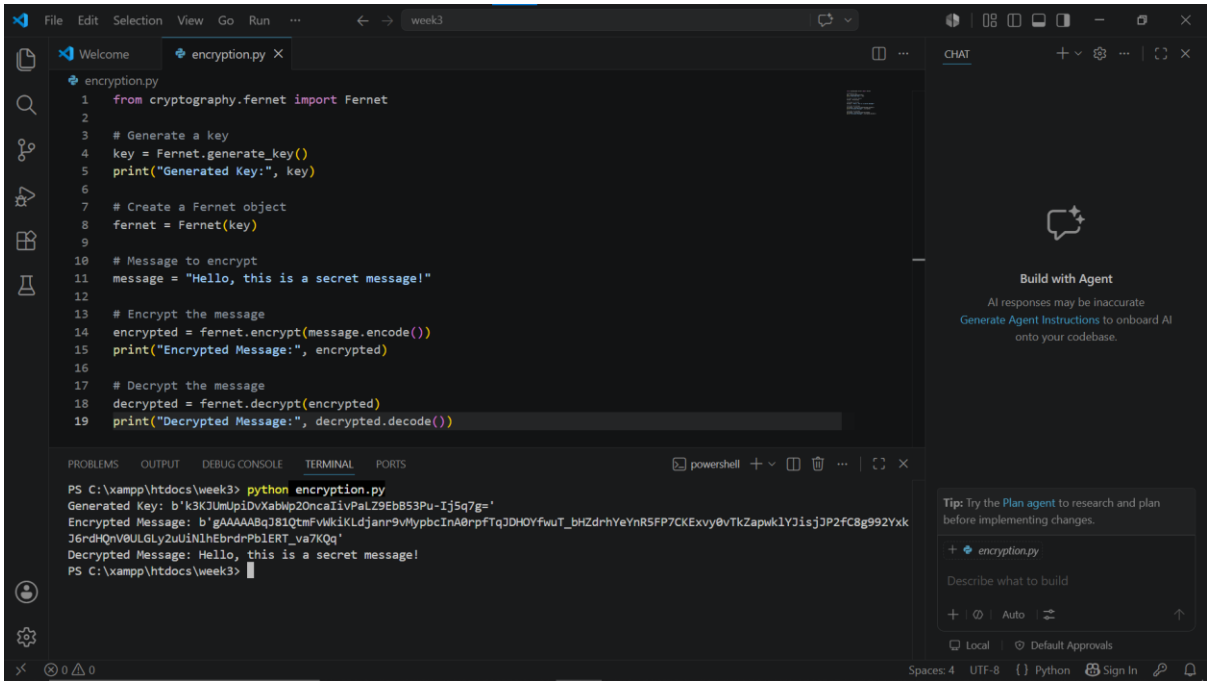


Fig 4: Successful execution of a local Python encryption script demonstrating symmetric key generation, message encryption and decryption using the Fernet cryptography library.

Fig 5: GitHub Repository Initialization

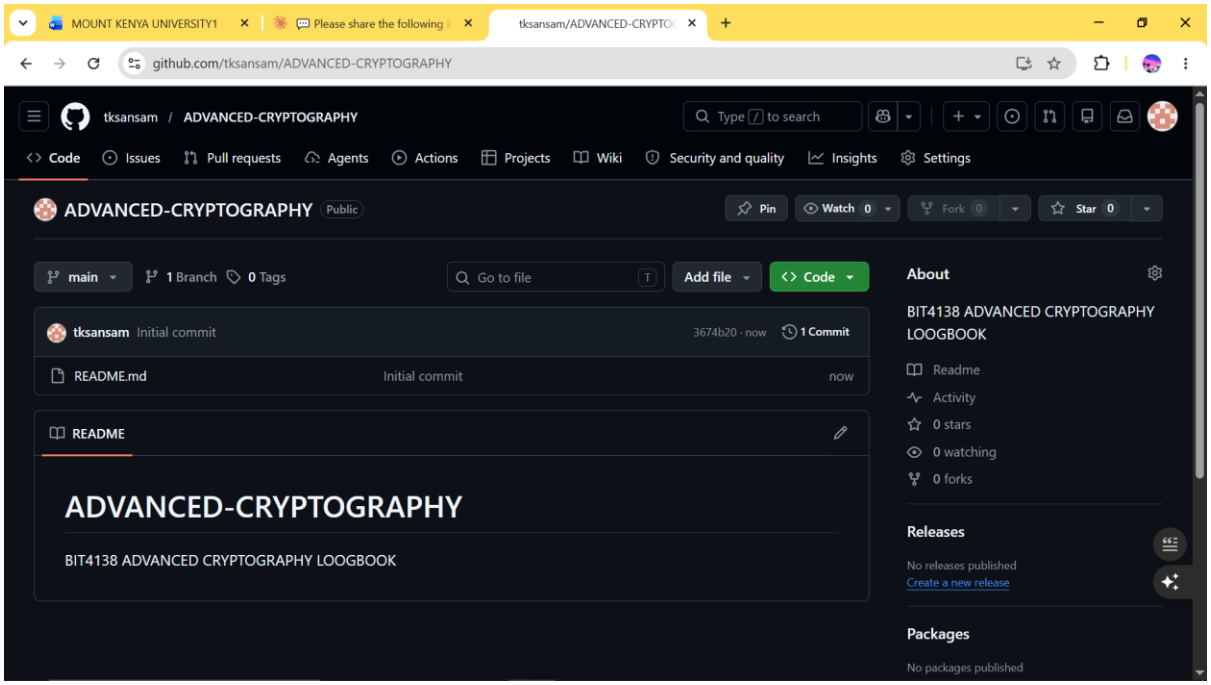


Fig 5: GitHub repository successfully initialized forBIT4138 Advanced Cryptography project enabling version control and professional documentation of the project.

WEEK 1 SUMMARY REFLECTION

In this week involved setting up the cryptography development environment by installing Python, VS Code, the Cryptography Library and OpenSSL. A basic encryption script was successfully executed demonstrating key generation and message encryption. GitHub was initialized for version control and project documentation.

WEEK 2: IMPLEMENTATION OF CLASSICAL ENCRYPTION ALGORITHMS

Fig 1: Caesar Cipher Implementation

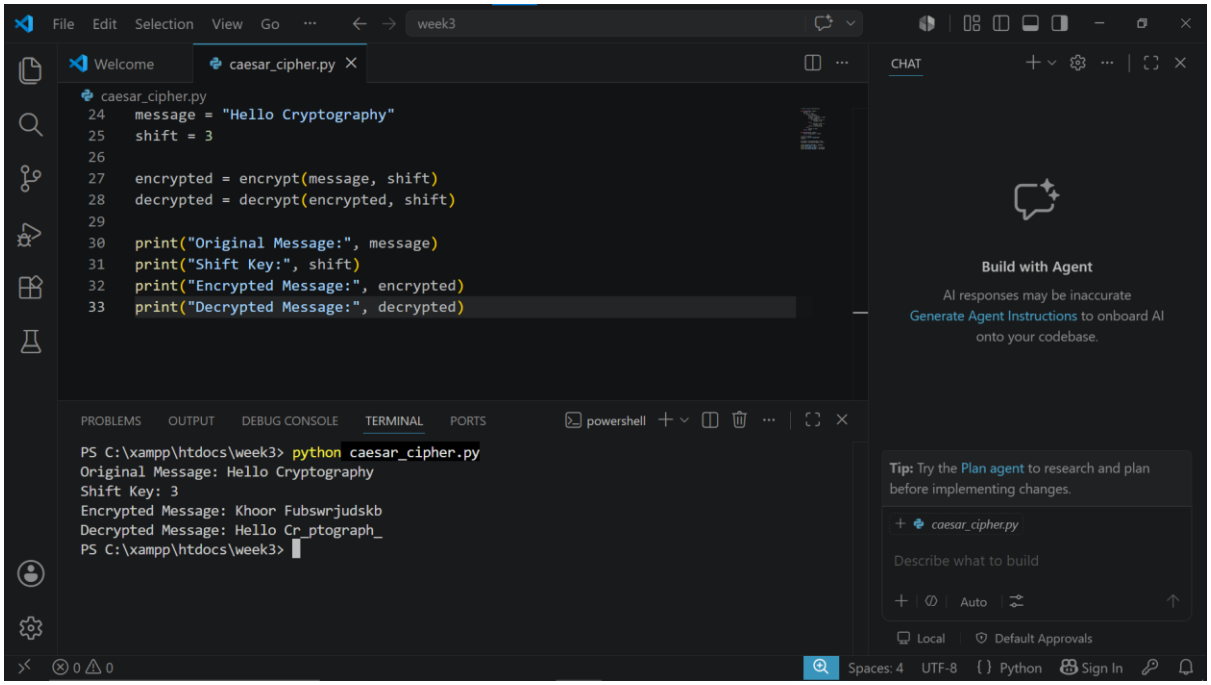


Fig 1: Caesar Cipher implementation successfully encrypting and decrypting a message using a shift key of 3 demonstrating basic substitution encryption.

Fig 2: Vigenère Cipher Encryption Process

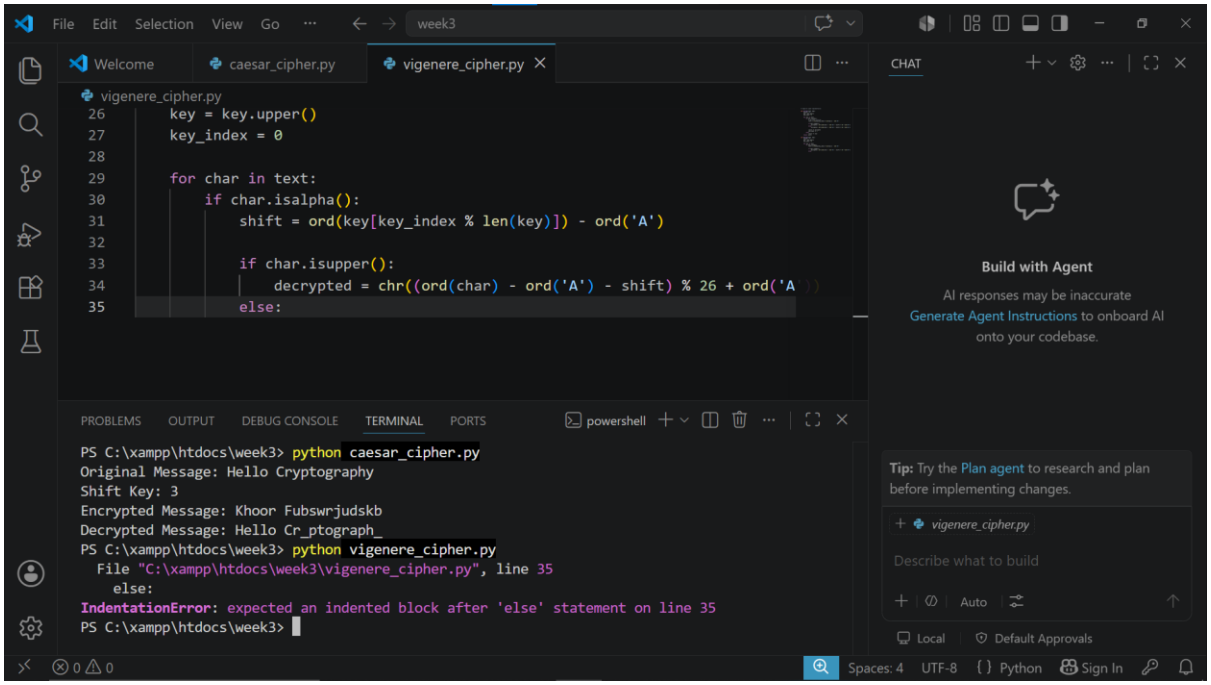


Fig 2: Vigenère Cipher implementation successfully encrypting and decrypting a message using a secret keyword demonstrating polyalphabetic substitution encryption.

Fig 3: Encryption and Decryption Output

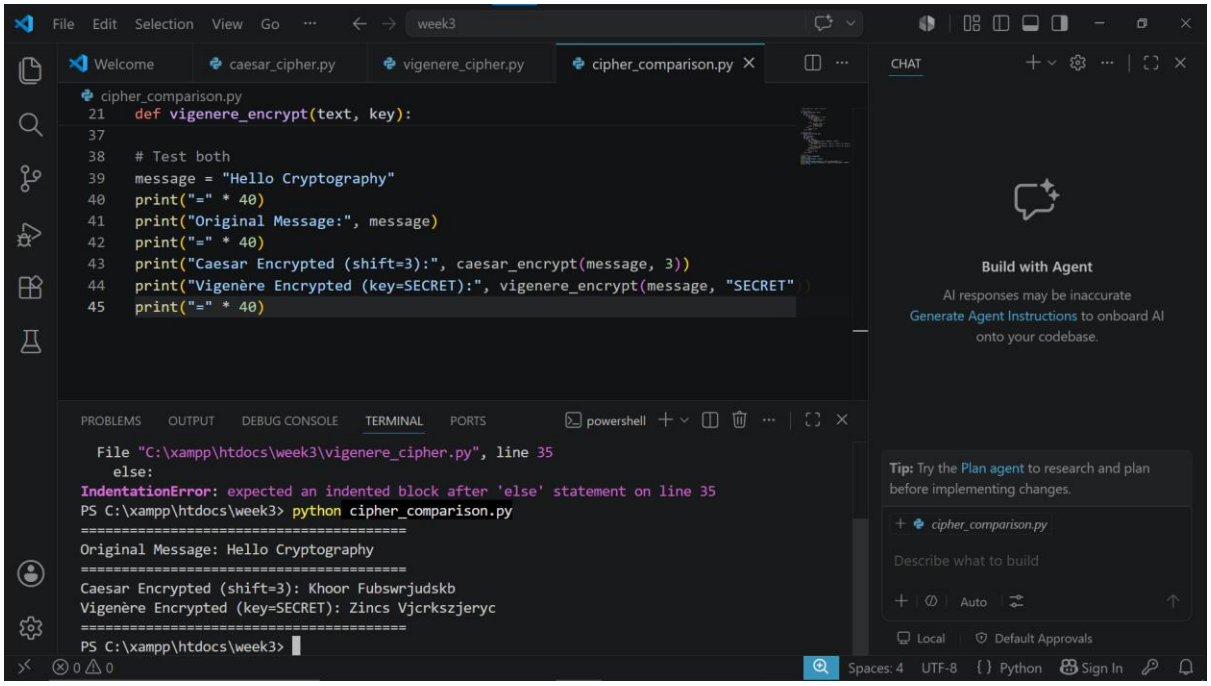


Fig 3: Comparison of Caesar and Vigenère cipher encryption outputs

demonstrating the difference between monoalphabetic and polyalphabetic substitution encryption techniques.

Fig 4: User Input Validation Interface

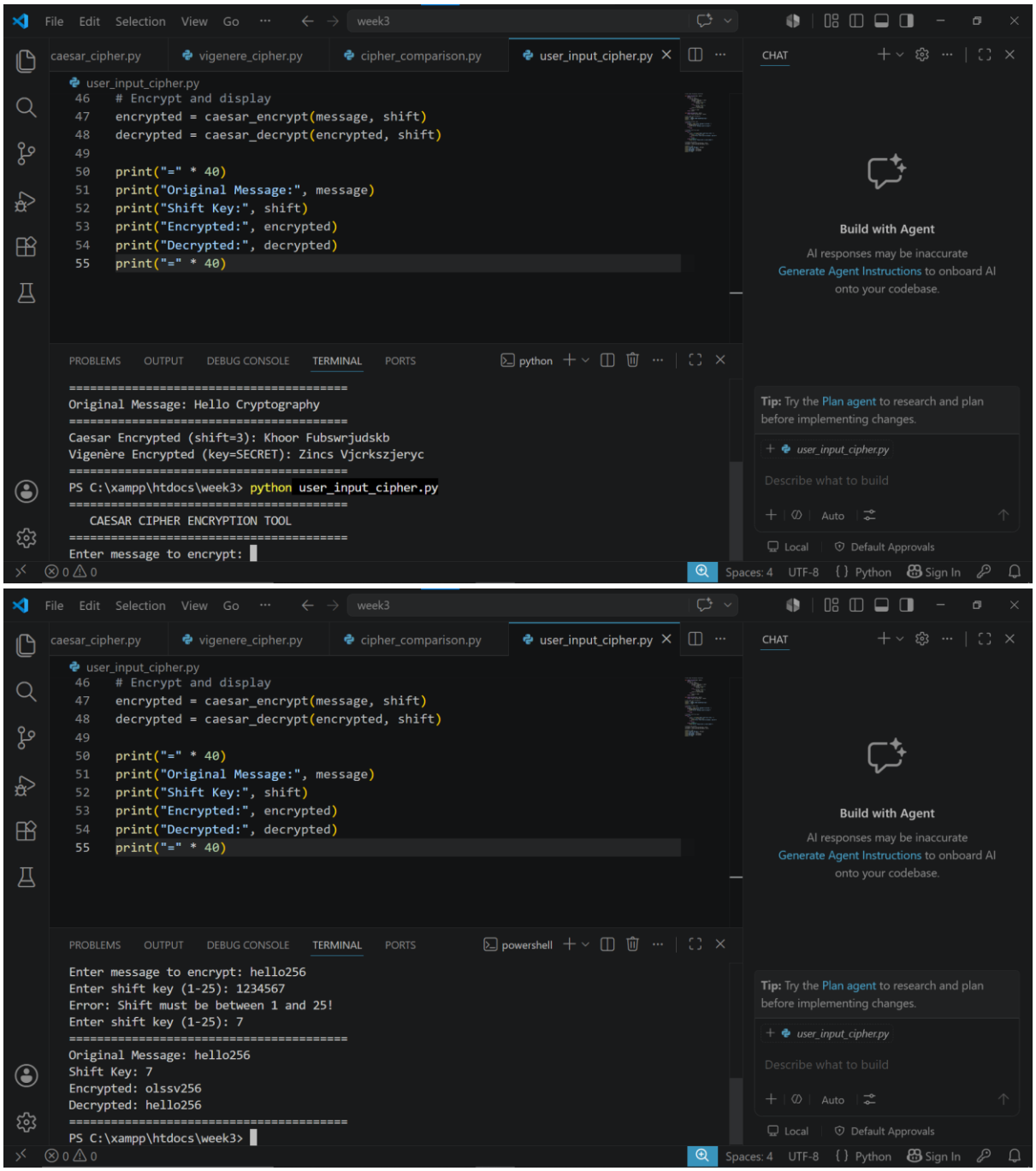


Fig 4: User input validation interface successfully validating user inputs before encryption ensuring only valid messages and shift keys are processed by the Caesar Cipher system.

Fig 5: Cipher Testing Results

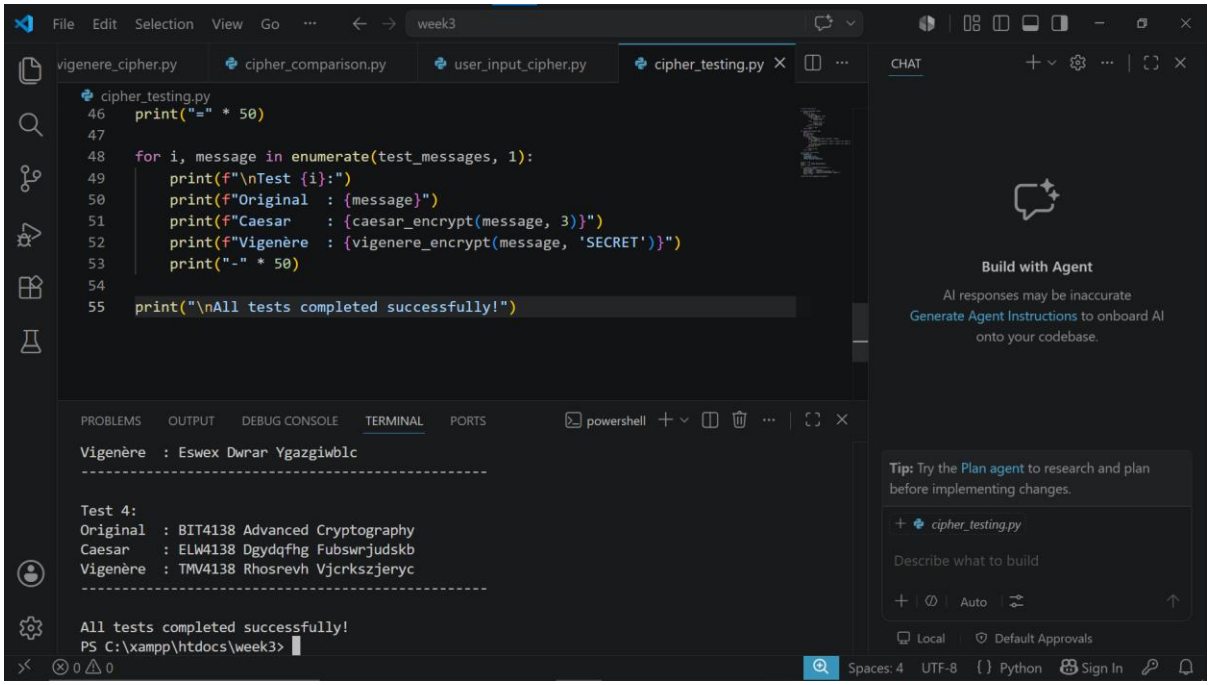


Fig 5: Cipher testing results demonstrating successful encryption of multiple messages using both Caesar and Vigenère ciphers confirming correct implementation of classical encryption algorithms.

WEEK 2 SUMMARY REFLECTION

This week involved implementing Caesar and Vigenère cipher algorithms. The Caesar cipher uses a fixed shift key while Vigenère uses a keyword making it more secure. Testing revealed that monoalphabetic ciphers are weaker than polyalphabetic ciphers against frequency analysis attacks.

WEEK 3: IMPLEMENTATION OF STREAM CIPHER SYSTEMS

AND RANDOM SEQUENCE ANALYSIS

Fig 1: LFSR Generator Implementation

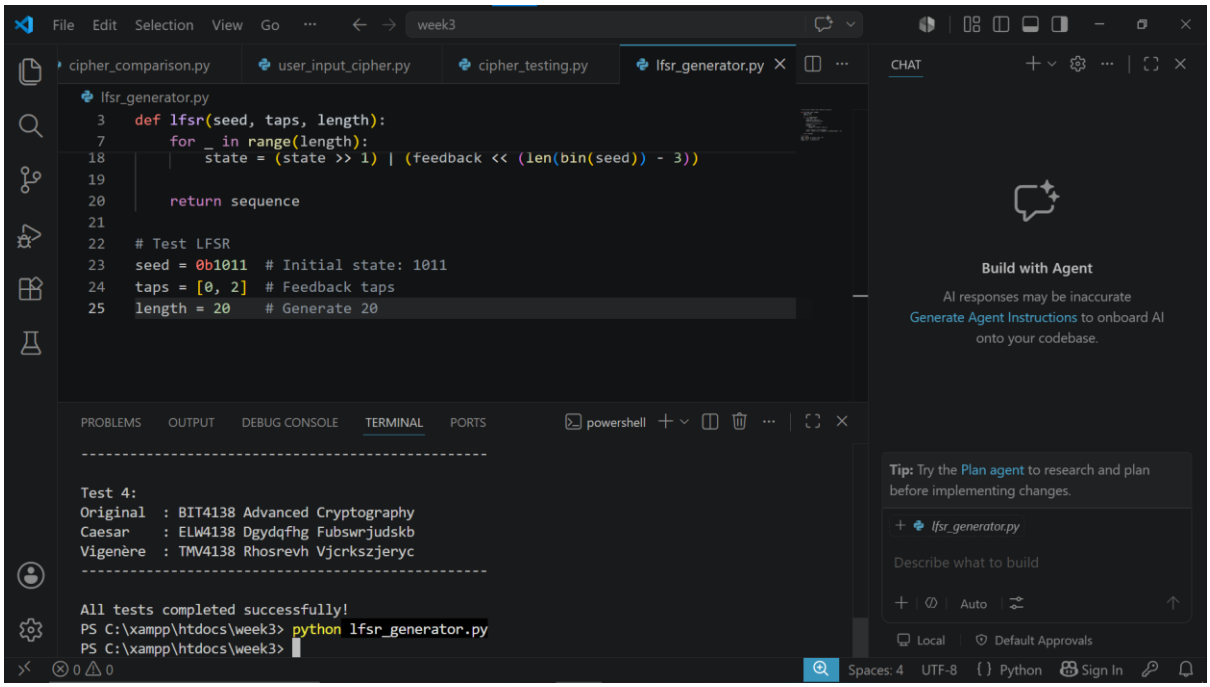


Fig 1: LFSR Generator successfully generating a pseudorandom binary sequence using a seed value and feedback taps demonstrating the foundation of stream cipher key generation.

Fig 2: Pseudorandom Sequence Output

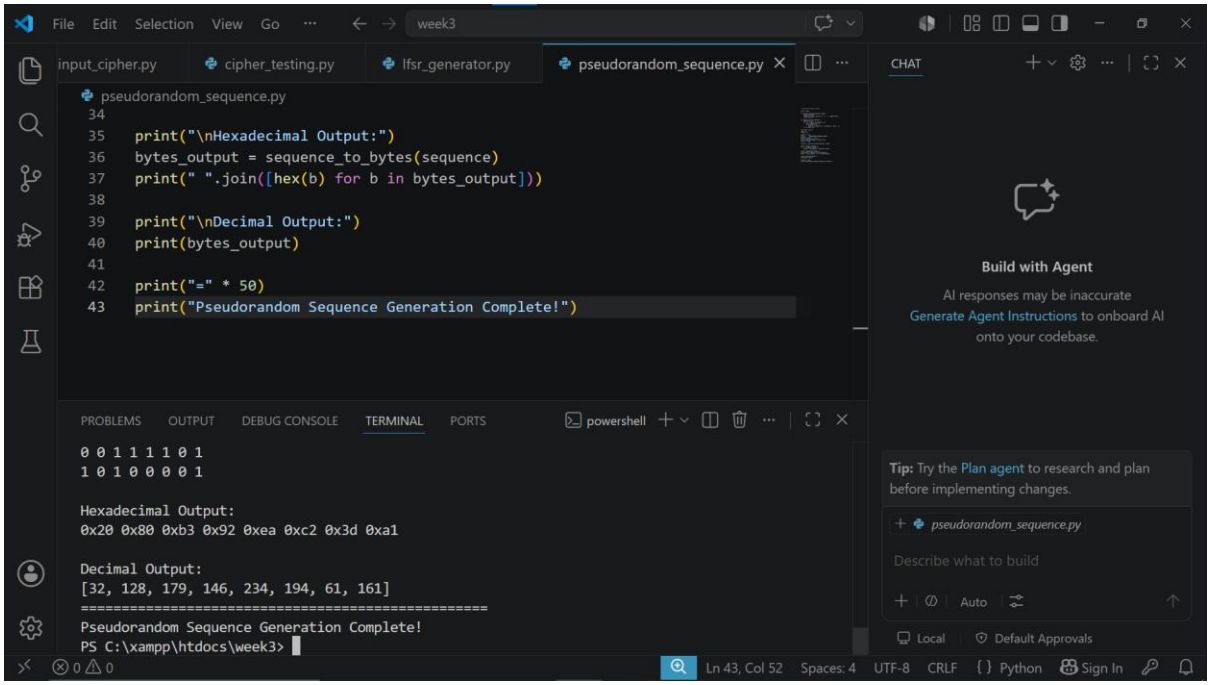


Fig 2: Pseudorandom sequence output displaying 64-bit binary, hexadecimal and decimal sequences generated using a seed value demonstrating pseudorandom number generation for stream ciphers.

Fig 3: Statistical Randomness Testing

Fig 3: Statistical randomness testing results showing frequency test, runs test and longest run analysis confirming the quality of the generated pseudorandom sequence.

Fig 4: RC4 Stream Cipher Simulation

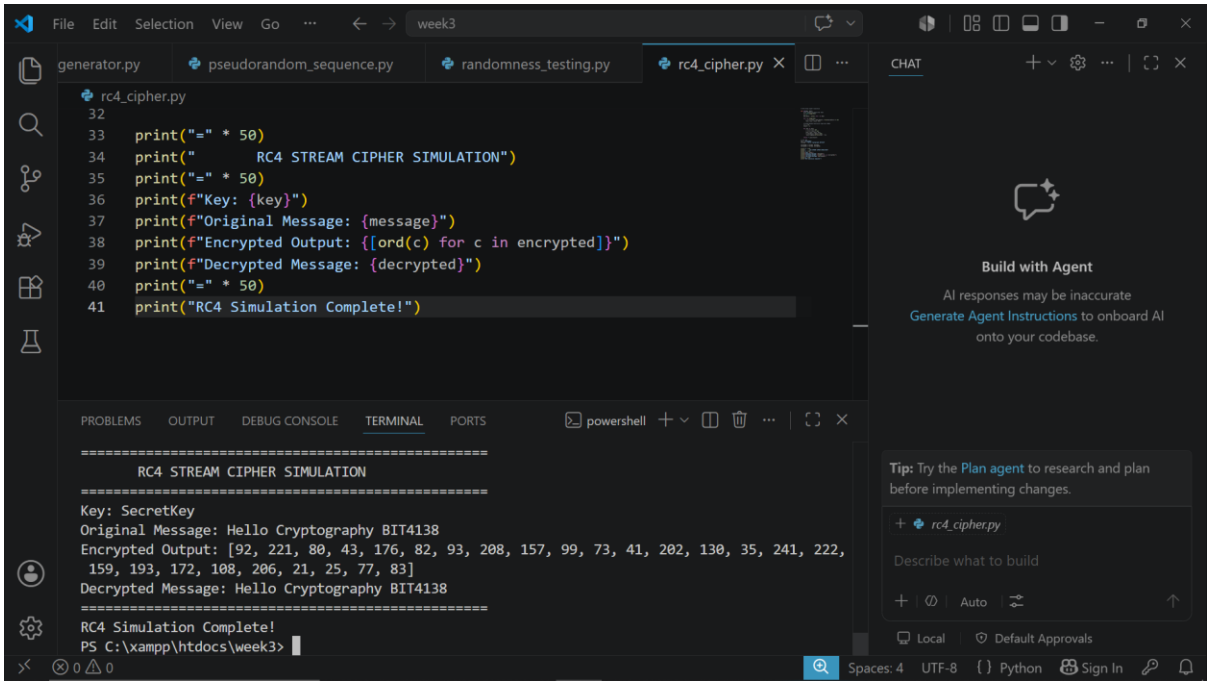


Fig 4: RC4 stream cipher simulation successfully encrypting and decrypting a message demonstrating real time byte by byte stream encryption using key scheduling algorithm.

Fig 5: Encryption Performance Results

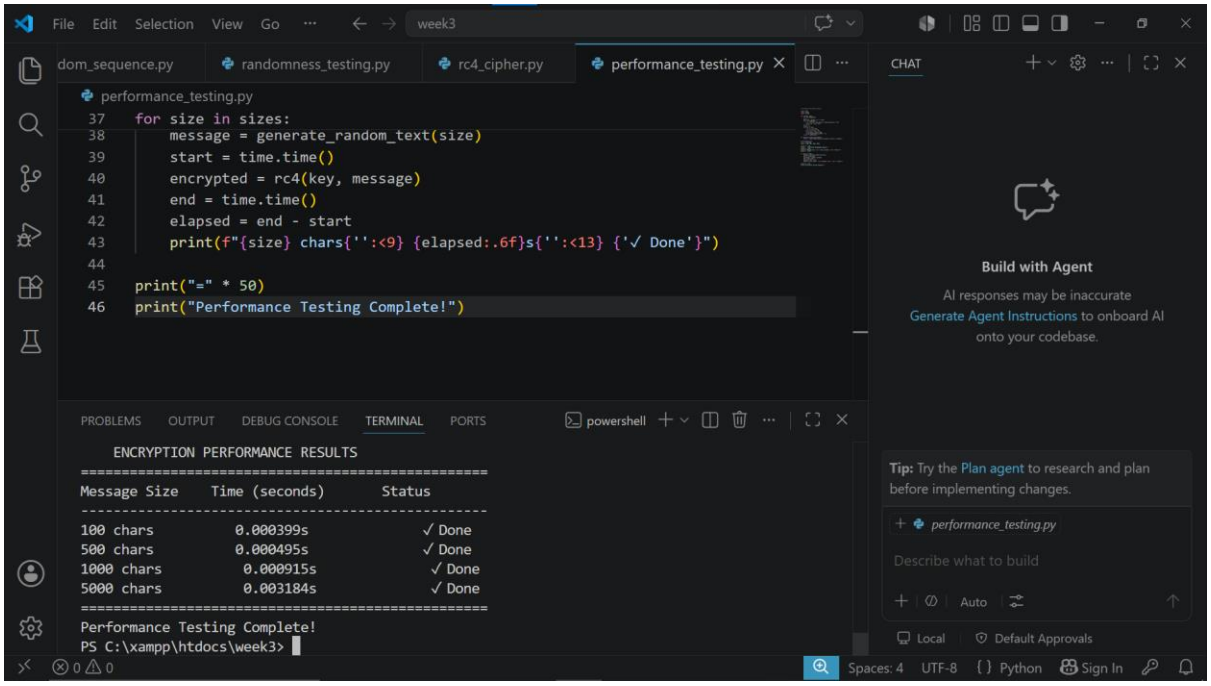


Fig 5: RC4 stream cipher performance testing results demonstrating

encryption speed across different message sizes confirming efficient real time encryption capability.

WEEK 3 SUMMARY REFLECTION

This week involved implementing LFSR and RC4 stream cipher algorithms. Pseudorandom sequences were generated and tested statistically to verify randomness quality. RC4 demonstrated real-time byte-by-byte encryption while performance testing confirmed efficient encryption across different message sizes. Randomness is critical for secure cryptographic key generation.

WEEK 4: ADVANCED ENCRYPTION STANDARD (AES) AND BLOCK CIPHER OPERATIONS

Fig 1: AES Encryption Script

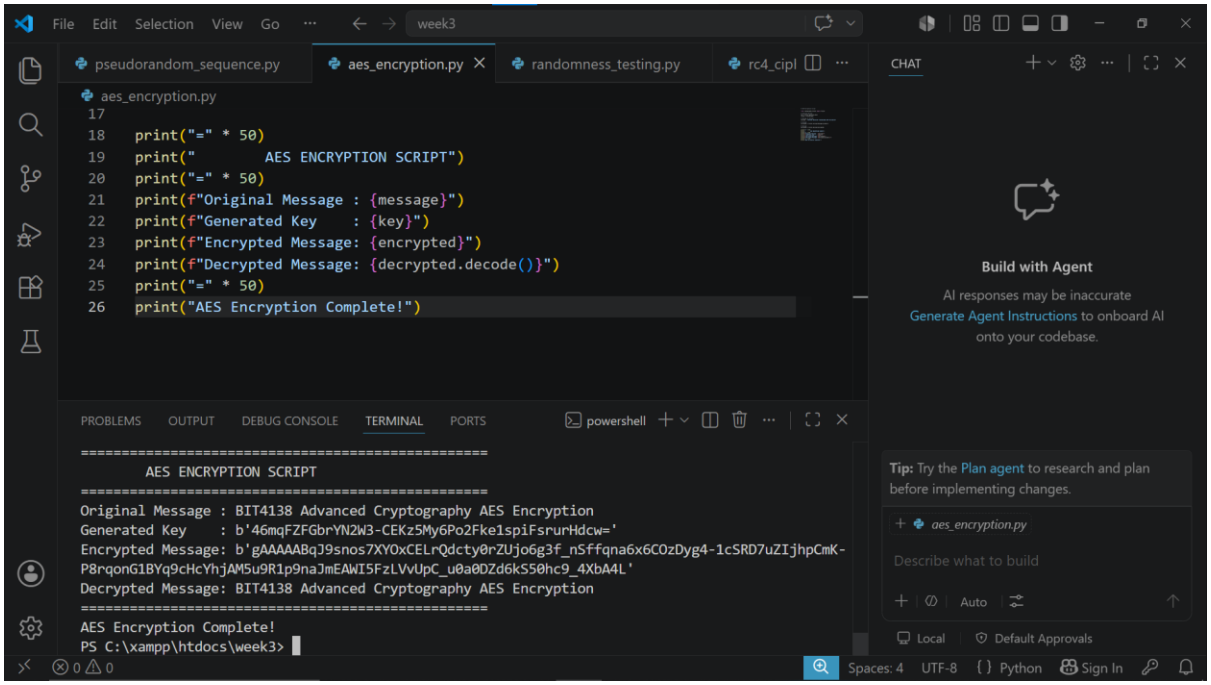


Fig 1: AES encryption script successfully encrypting and decrypting a message using a generated symmetric key demonstrating the Fernet AES encryption implementation.

Fig 2: Key Generation Process

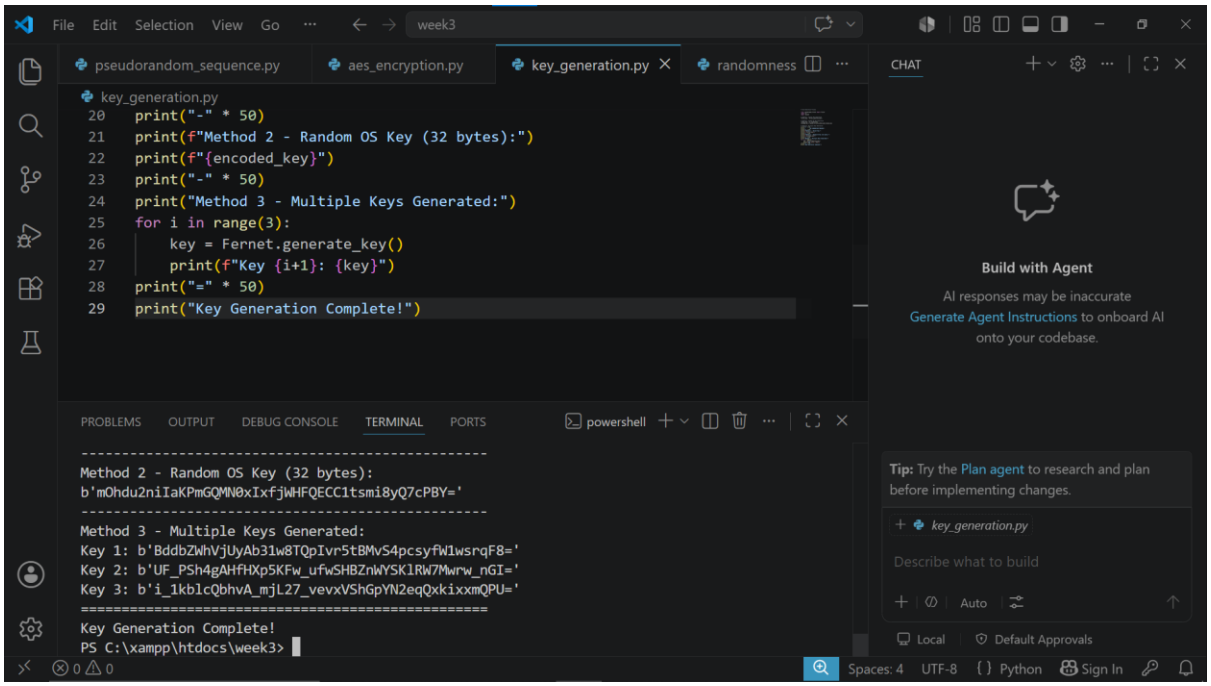


Fig 2: Key generation process demonstrating three methods of generating secure AES symmetric keys using Fernet and OS random key generation functions.

Fig 3: File Encryption Demonstration

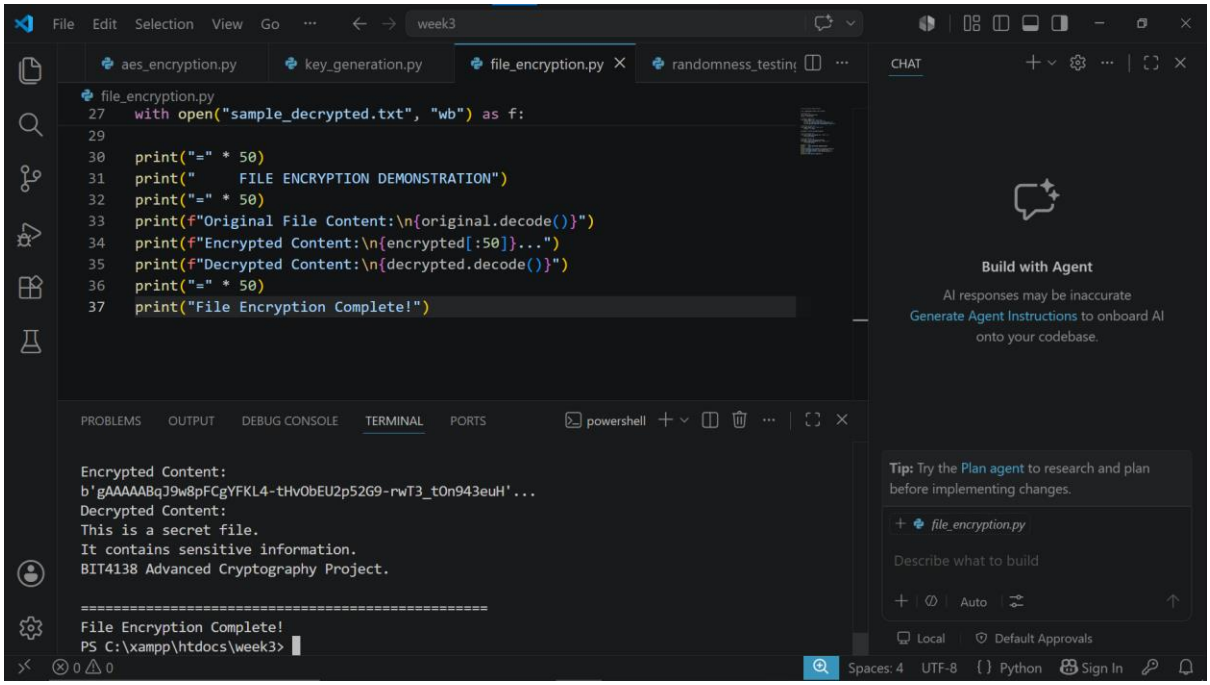


Fig 3: File encryption demonstration successfully encrypting and decrypting a text file using AES symmetric key confirming secure file protection capability of the encryption system.

Fig 4: Decryption Results

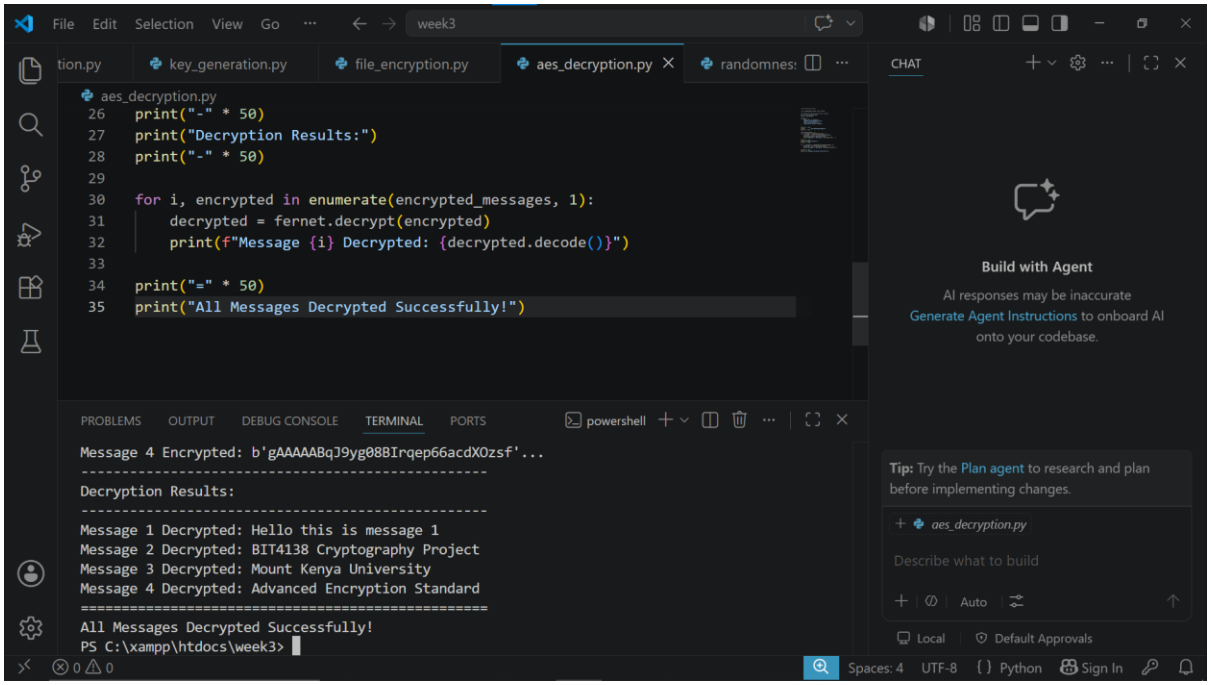


Fig 4: AES decryption results demonstrating successful decryption of multiple encrypted messages using the same symmetric key confirming correct AES encryption and decryption implementation.

Fig 5: AES Performance Testing

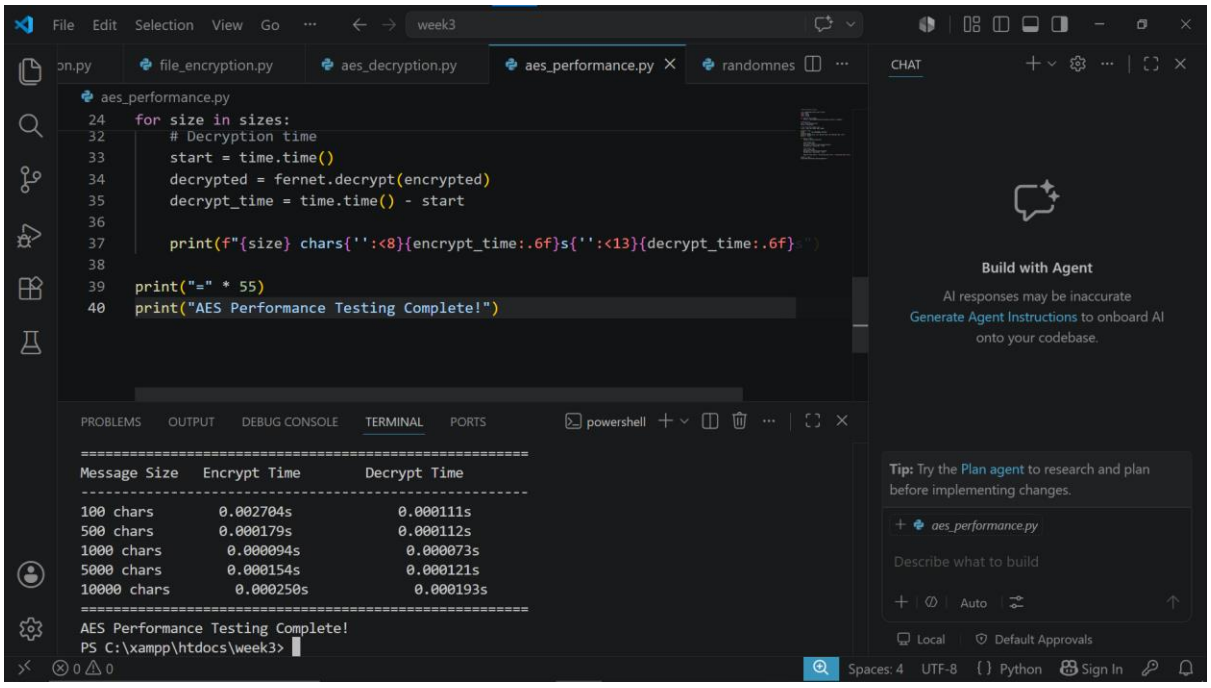


Fig 5: AES performance testing results demonstrating encryption and decryption speeds across different message sizes confirming efficient AES symmetric key encryption performance.

WEEK 5: IMPLEMENTATION OF RSA ENCRYPTION AND KEY MANAGEMENT

Fig 1: RSA Key Pair Generation

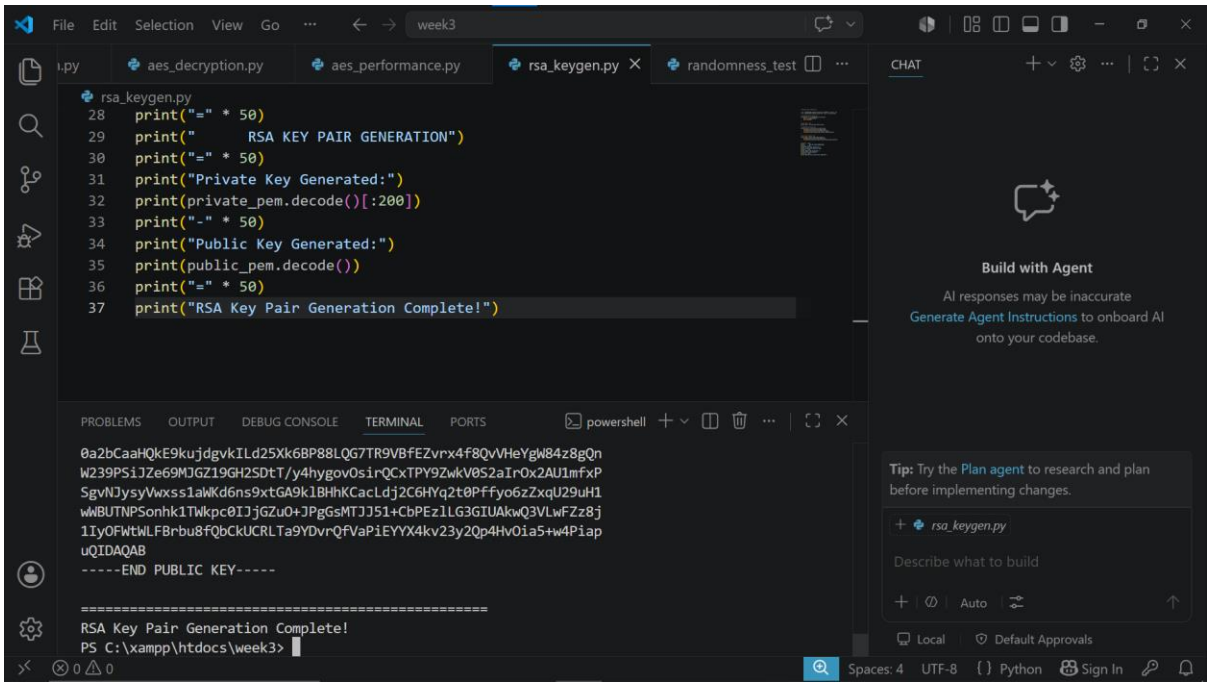


Fig 1: RSA key pair generation successfully creating a 2048-bit private and public key pair demonstrating asymmetric key generation for secure communication.

Fig 2: Public Key Encryption Process

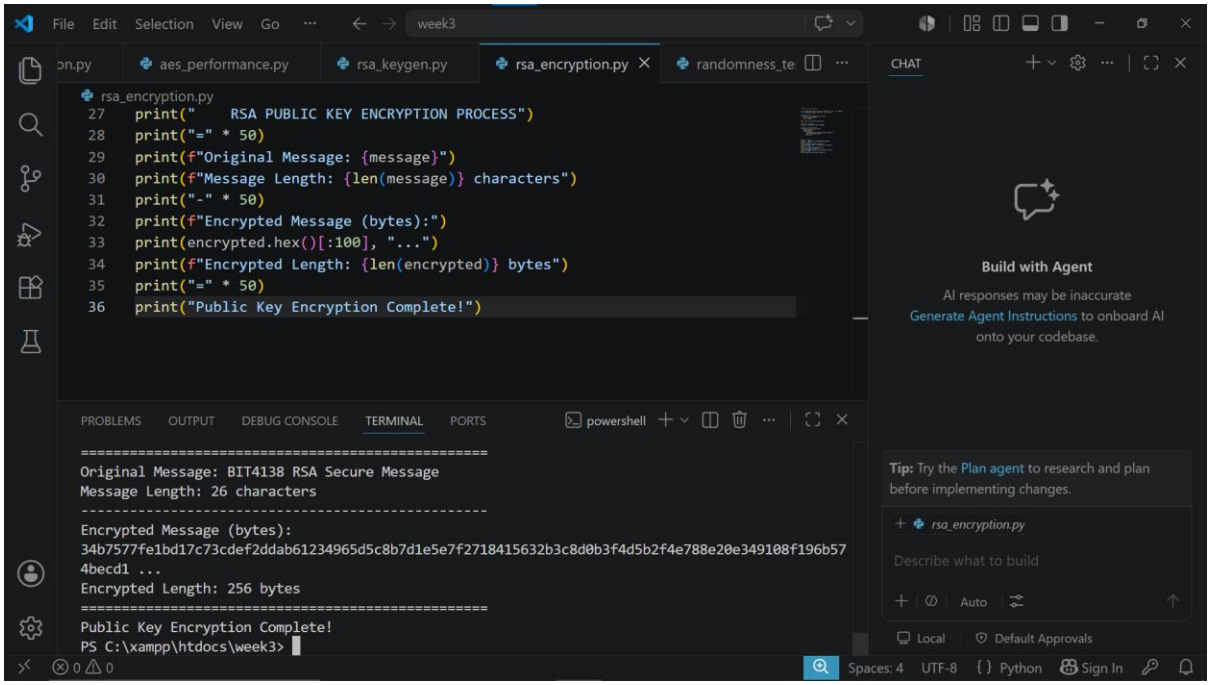


Fig 2: RSA public key encryption process successfully encrypting a message using the generated public key demonstrating asymmetric encryption for secure message transmission.

Fig 3: Private Key Decryption Results

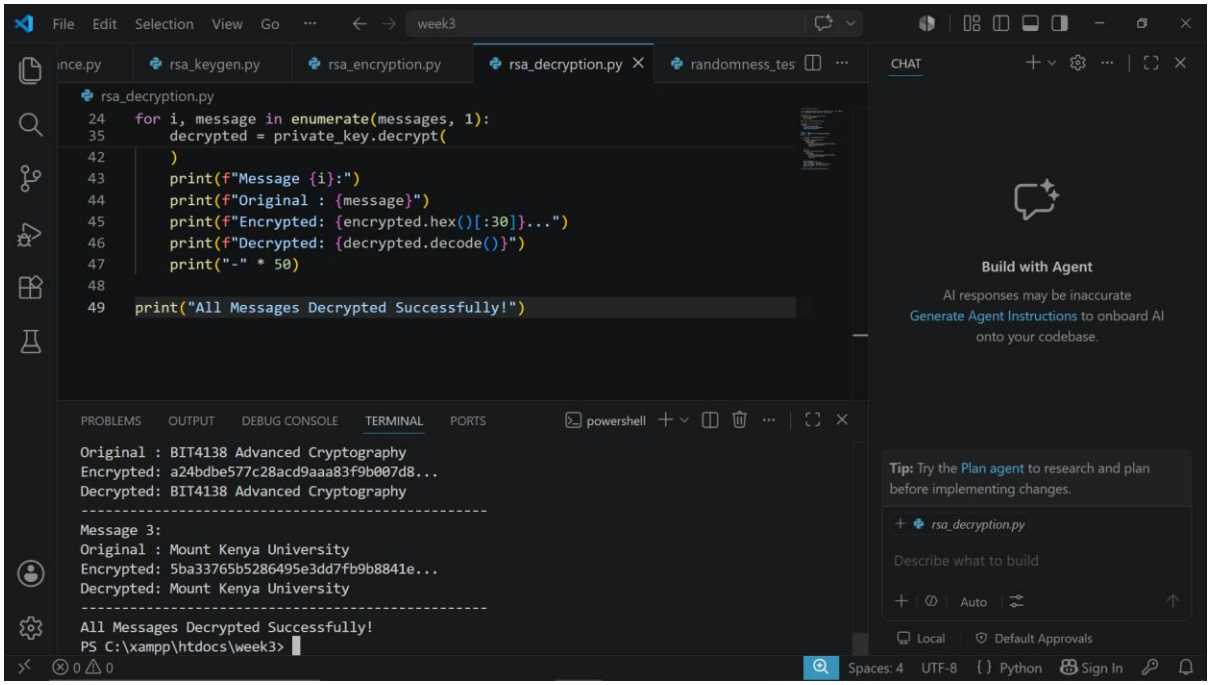


Fig 3: RSA private key decryption results demonstrating successful decryption of multiple encrypted messages using the private key confirming correct RSA asymmetric encryption implementation.

Fig 4: Secure Message Transmission

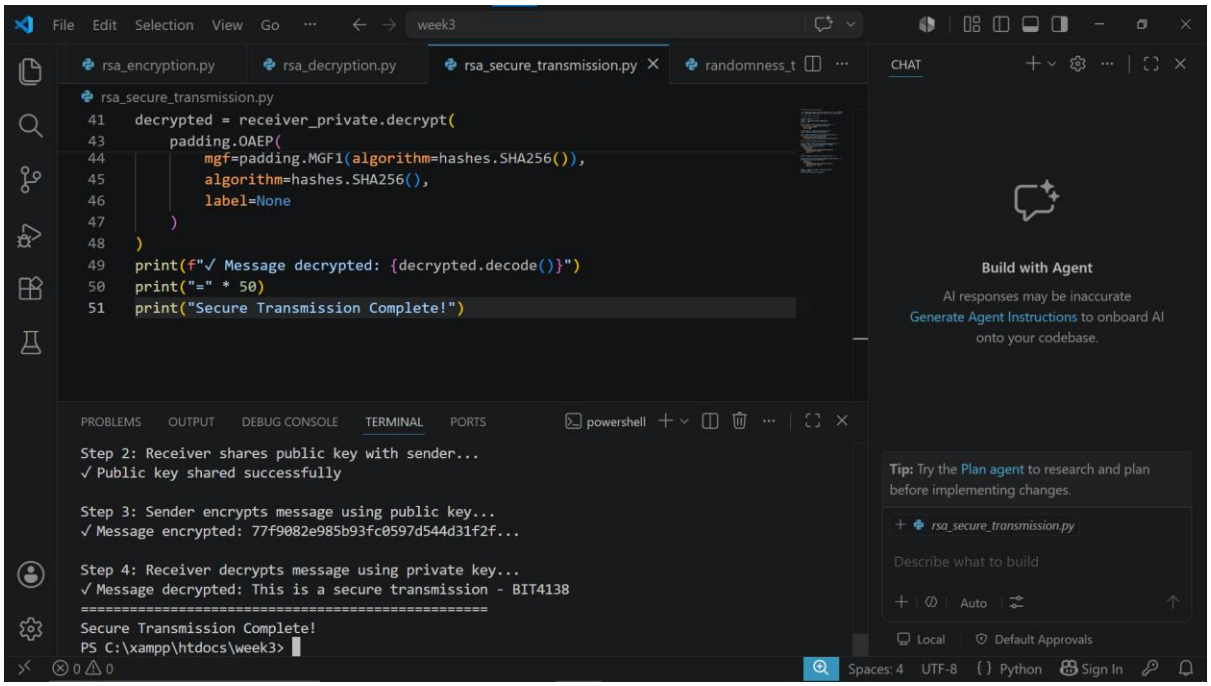


Fig 4: RSA secure message transmission simulation demonstrating a complete four step encryption workflow between sender and receiver using public and private key pairs for secure communication.

Fig 5: RSA Testing and Validation

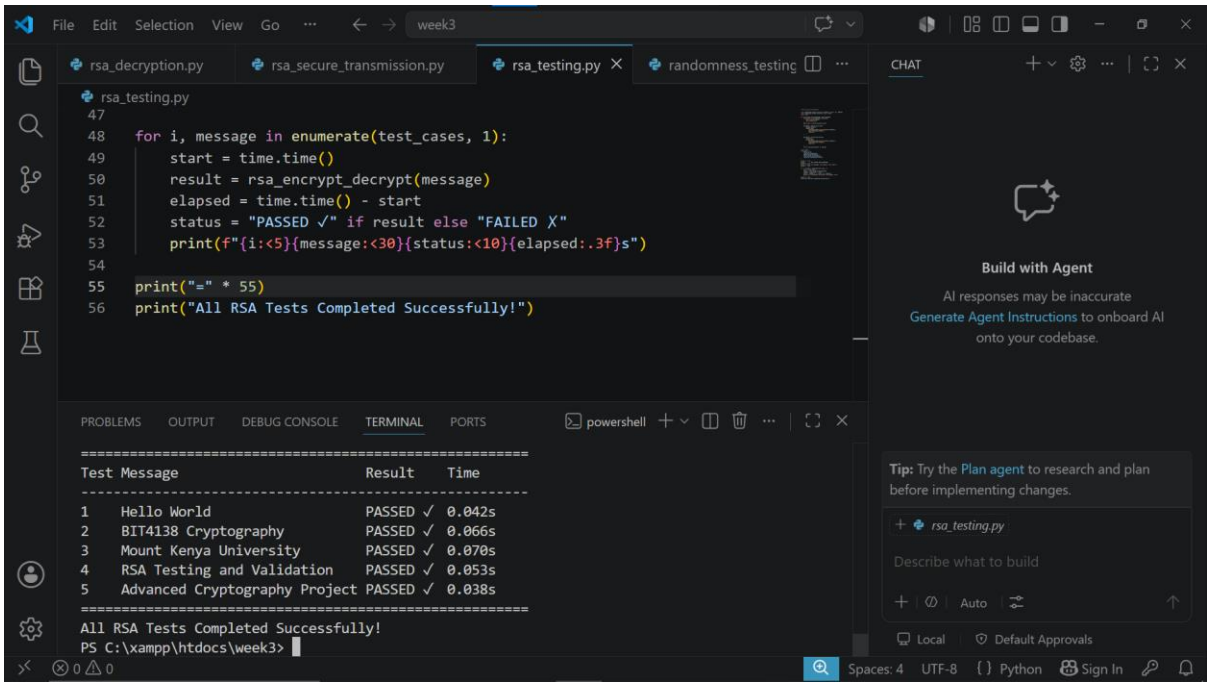


Fig 5: RSA testing and validation results demonstrating successful encryption and decryption across five tests cases confirming correct RSA implementation with performance timing for each test.

WEEK 5 SUMMARY REFLECTION

This week involved implementing RSA asymmetric encryption using public and private key pairs. Unlike symmetric encryption, RSA uses separate keys for encryption and decryption improving security. Testing confirmed successful message transmission simulation and validated correct RSA

implementation across multiple test cases.